RRRR: Rapid Real-World Residual RL for Multi-Task VLA Adaptation

Anonymous RAL Submission

***Abstract*— Vision-Language-Action (VLA) foundation models generalize across a broad range of manipulation tasks, yet reliable real-world deployment often requires further adaptation to task-specific dynamics. We study two complementary post-training settings: rapid residual adaptation of independently task-tuned policies and consolidation of task-specific interaction experience into one shared multi-task model. To this end, we propose RRRR (Rapid Real-World Residual RL), a plug-and-play framework that couples task-specific residual adaptation of frozen VLA backbones with residual-guided consolidation of successful rollouts into a standalone shared VLA. For task-specific adaptation, a high-frequency residual policy provides closed-loop corrections to low-frequency VLA action chunks, is optimized with a unified imitation learning (IL) and reinforcement learning (RL) objective, and can incorporate auxiliary feedback such as force feedback and estimated depth maps. For shared multi-task consolidation, task-specific residual policies are trained on a frozen mixed-task SFT backbone, and the executed residual scales recorded along successful rollouts are used as sample-level weights when fine-tuning the shared VLA. On four real-world manipulation tasks, residual adaptation raises the average success rates of task-specific OpenVLA-OFT and $\pi_{0.5}$ backbones from 0.00% and 45.00% to 91.25%, respectively; the evaluated checkpoints are reached after 3.15–27.30 min of residual training in our setup. Weighted consolidation attains 85.00% average success, compared with 23.75% when the same rollouts are trained with uniform weights. On a custom 25-task LIBERO joint-training setting, RRRR reaches 91.80% average success, exceeding two representative RL-based data-generation baselines by 6.64 and 4.28 percentage points.**

I. Introduction

In recent years, Vision-Language-Action (VLA) foundation models have enabled cross-task generalization through large-scale pretraining, yet reliable real-world deployment remains challenging due to dynamics discrepancies, sensor noise, and contact-induced perturbations. Consequently, task-specific post-training is often required.

Post-training poses two distinct deployment challenges. Independently task-tuned VLAs may still require alignment to deployment-specific dynamics, whereas a shared multi-task VLA must reconcile heterogeneous task distributions and competing optimization signals within one model. Directly updating a large VLA during real-world interaction is also operationally costly.

We first address rapid single-task adaptation by viewing it as *residual compensation* rather than further modification of the base model. For each task, RRRR (Rapid Real-World Residual RL) attaches a trainable residual module to a policy independently adapted to that task via supervised fine-tuning (SFT), whose parameters remain frozen during residual learning. A fast–slow architecture combines low-frequency semantic actions from the frozen backbone with high-frequency residual corrections.

RRRR further employs a unified IL–RL objective that uses demonstrations and online human corrections to guide early residual learning while progressively increasing the contribution of RL. The residual module can additionally consume feedback not used by the base VLA, including force feedback and estimated depth maps.

We then extend this mechanism to shared multi-task adaptation. Starting from a backbone initialized by mixed-task SFT, we train task-specific residual policies, collect successful composed-policy rollouts, and use the recorded residual scales to guide a weighted SFT update of the shared model.

Experiments across four real-world manipulation tasks evaluate rapid residual adaptation, residual-design ablations, and shared-model consolidation, while a custom 25-task LIBERO study evaluates the multi-task procedure at a larger task scale.

In summary, our contributions are three-fold:

1) We propose a plug-and-play fast–slow architecture in which a lightweight high-frequency residual policy closes the control loop around a frozen VLA backbone and is trained with a unified IL–RL objective for stable, short-wall-clock real-world adaptation.
2) We design a dynamic residual scale that gates corrective intervention during execution; its recorded value provides a per-transition proxy for executed residual-intervention strength.
3) We develop residual-guided weighted SFT, which converts the recorded scales into curriculum-style sample weights and consolidates successful base–residual rollouts into one standalone shared multi-task VLA.

Overall, RRRR connects rapid task-specific residual adaptation with shared multi-task experience consolidation. Figure 1 provides an overview of the proposed framework.

II. Related Work

A. VLA Models and Post-Training Adaptation

Recent VLA models [1], [2], [3], [4], [5], [6] achieve cross-task generalization through large-scale pretraining but still require post-training to adapt to specific environments. RT-2 [2] transfers vision–language pretraining to robot control, while OpenVLA [3] and $\pi_{0.5}$ [5] explore scalable generalist manipulation models. Commercial wheeled bimanual platforms such as ZERITH H1 [7] further broaden the embodiment space for mobile service manipulation. Rapid task-specific correction and joint consolidation of task-level experience into a shared model remain distinct practical challenges.

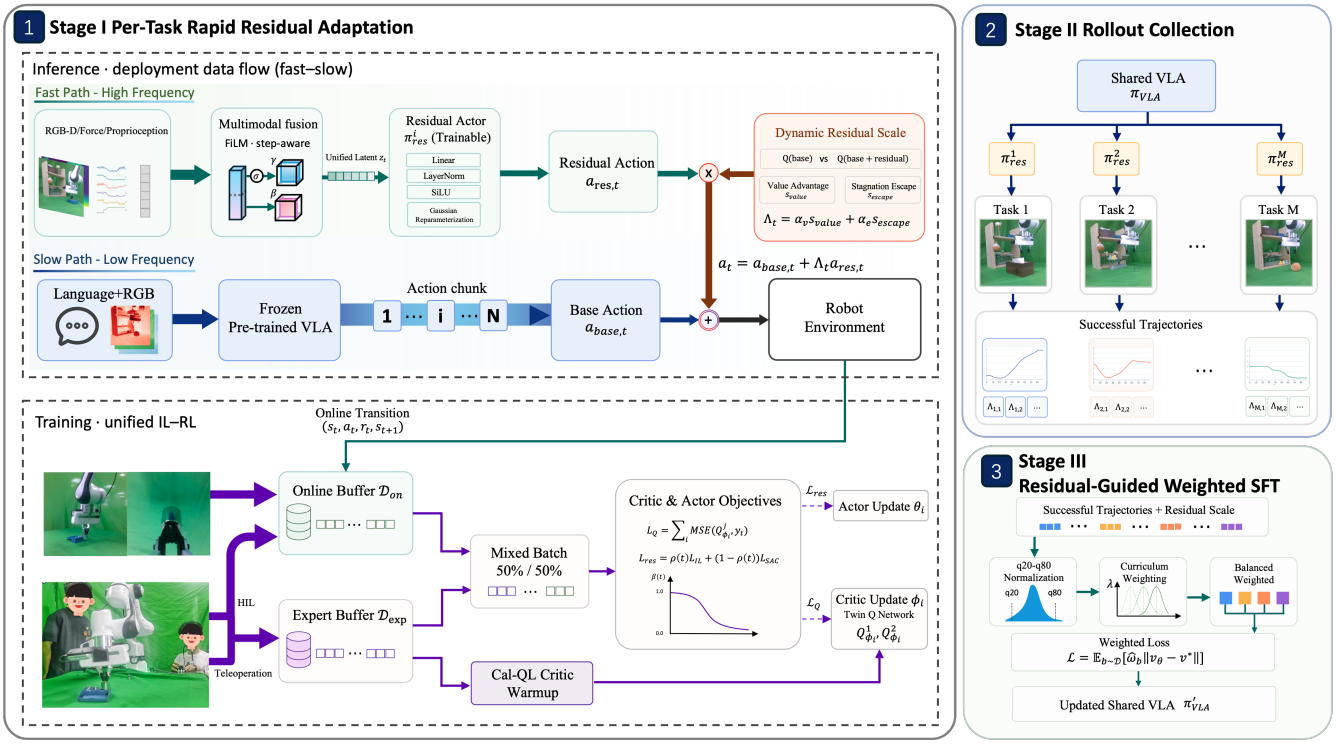


Fig. 1. Overview of RRRR. **Stage I** trains task-specific residual policies around frozen VLAs with unified IL–RL. **Stage II** collects successful composed-policy rollouts and their executed residual scales on a shared mixed-SFT VLA. **Stage III** uses the recorded scales to weight SFT of the shared VLA.

B. Residual Policy

Residual corrections on a frozen base policy have proven effective for fine-grained manipulation: ResiP [8] for precision assembly, and ResFiT [9] with sample-efficient off-policy RL. In contrast, our work develops a VLA-oriented framework where a fast–slow architecture enables rapid residual adaptation while residual signals guide shared multi-task consolidation.

C. Human Correction and Stable Real-World RL

Real-world RL is often limited by costly data collection and unstable exploration. Prior work incorporates demonstrations or human-in-the-loop (HIL) corrections to improve safety and sample efficiency. CR-DAgger [10] enables incremental human corrections, while HIL-SERL [11] combines demonstrations with online learning to improve coverage and performance.

D. Residual Learning for Data Generation

Recent work uses RL to generate higher-quality training data for generalist policies. RLDG [12] improves policy training through RL-generated trajectories, while the diffusion-policy RL pipeline of Yang et al. [13], denoted DPRL for brevity, generates smooth, low-variance trajectories for subsequent VLA training. PLD [14] collects base–residual rollouts via distribution-aware replay and distills them into VLA models with standard SFT. In contrast, we reuse the recorded residual scales as transition-level weights when updating a shared VLA backbone, enabling balanced joint multi-task consolidation.

III. Methods

A. Problem Formulation

We model multi-task robotic control as an MDP $(\mathcal{S}, \mathcal{A}, \mathcal{P}, \mathcal{R}, \gamma_{\text{disc}}, \mathcal{T})$, with a distribution $\mathcal{T}$ over M tasks $\{T_i\}_{i=1}^M$. For notational simplicity, s_t denotes the observation available to the residual controller rather than an unobserved latent state. It comprises RGB-D observations I_t, proprioception p_t, the language instruction l_i, and force–torque measurements $[f_t, \tau_t]$, where the depth channel is estimated from RGB using Depth Anything V2 [15]. The agent executes a continuous action $a_t \in \mathcal{A}$ under dynamics $\mathcal{P}(s_{t+1} \mid s_t, a_t)$ and receives task-specific reward r_t. The base VLA retains its native observation interface; the depth and force channels are consumed only by the residual path.

We use Φ_{base} to denote the frozen VLA parameters under the setting being considered. In rapid single-task adaptation, $\Phi_{\text{base}} = \Phi_i^{\text{SFT}}$ denotes the parameters of the VLA independently fine-tuned for task T_i. In multi-task adaptation, $\Phi_{\text{base}} = \Phi_{\text{mix}}^{\text{SFT}}$ denotes the parameters of one shared VLA π_{VLA} trained on mixed demonstrations from all tasks.

For task T_i, the base and residual actions are combined using the smoothed scale $\bar{\Lambda}_{i,t}^{\text{act}} \in [0, \lambda_{\max}]$, as defined in eq. (3). The scale is initialized to λ_{init} and updated after evaluating the current scaled residual candidate, as described in section III-C. A task-specific critic $Q(s, a; \phi_i)$ evaluates the composed action.

In the single-task setting, residual learning is performed independently for each task-specific SFT VLA on $\mathcal{D}^i = \mathcal{D}^i_{\text{exp}} \cup \mathcal{D}^i_{\text{on}}$, which pools expert demonstrations and online

rollouts of task T_i. In the multi-task setting, all residual policies share the same frozen mixed-SFT VLA. After optimizing the task-specific residual actors and critics, we collect successful composed-policy rollouts and use their residual-derived weights to update the shared VLA, as detailed in sections III-D and III-F.

B. Residual Policy for Real-World VLA Adaptation

Fast–Slow Residual Inference Architecture. To address the precision-latency trade-off in VLA deployment, we propose a fast–slow inference architecture. While large-scale VLA models can generate coherent action chunks, they inherently operate in an *open-loop* manner within each chunk, accumulating execution errors and failing to react to high-frequency haptic or dynamic disturbances. Our framework decouples policy inference into two synchronized paths: a global *Slow Path* and a reactive *Fast Path*.

Slow Path: Long-Horizon Planning. The slow path utilizes a pre-trained VLA backbone $\mu(s;\Phi_{\text{base}})$, which performs inference at a low frequency ($f_{\text{slow}} \approx 10\,\text{Hz}$), generating a temporal action chunk of length N:

$$\{a_{\text{base},t+n}\}_{n=0}^{N-1} = \mu(s_t;\Phi_{\text{base}}) \tag{1}$$

where N denotes the chunk size. These actions are cached into a first-in-first-out (FIFO) queue $\mathcal{Q}$, providing broad behavioral priors and a coarse reference trajectory for the robot.

Fast Path: Intra-Chunk-Conditioned Reactive Correction. At each control step t (with $f_{\text{fast}} \approx N \cdot f_{\text{slow}}$), the fast path retrieves the current reference $a_{\text{base},t}$ from $\mathcal{Q}$; when the queue is exhausted, a new base-action chunk is generated on demand (algorithm 1). A lightweight residual policy π_{res}, implemented as a multi-layer perceptron (MLP), computes the additive residual correction $a_{\text{res},t}$. To ensure the residual module is aware of its temporal progress within the current chunk, we feed it an *intra-chunk-conditioned* encoded state:

$$a_{\text{res},t} = \pi\big(\mathcal{E}(s_t, a_{\text{base},t}, n);\theta_i\big) \tag{2}$$

where $n \in \{0,\dots,N-1\}$ is the intra-chunk index within the action chunk and $\mathcal{E}$ is a Feature-wise Linear Modulation (FiLM)-based multimodal encoder (detailed in section III-E) that fuses the state, the base action, and the chunk index into a unified latent. For brevity, elsewhere we abbreviate the residual policy as $\pi(s_t;\theta_i)$, with the understanding that it acts on this encoded state, allowing for instantaneous error compensation.

Dynamic Action Composition. At control step t, let $\bar{\Lambda}_{i,t}^{\text{act}}$ denote the smoothed residual scale available to the actor before the current scale update. The command applied to the robot is

$$a_t = a_{\text{base},t} + \bar{\Lambda}_{i,t}^{\text{act}} a_{\text{res},t}. \tag{3}$$

The critic evaluates this same scaled base–residual action when computing the candidate advantage. The dynamic-scale mechanism described in section III-C subsequently updates the scale used by the next control step. This update order avoids an implicit circular dependence between the candidate advantage and the scale being computed. The resulting high-frequency closed loop allows the controller to respond to drift and unexpected contacts within a cached VLA action chunk.

The systematic training procedure for the residual policy and dynamic-scale update is detailed in algorithm 1.

Algorithm 1 Asynchronous Residual Policy Learning with Dynamic Scaling

Require: Frozen base-VLA params $\{\Phi_{\text{base}}^i\}_{i=1}^M$, expert datasets $\{\mathcal{D}_{\text{exp}}^i\}_{i=1}^M$
Ensure: Residual params $\{\theta_i,\phi_i\}_{i=1}^M$, rollout datasets $\{\mathcal{D}_{\text{on}}^i\}_{i=1}^M$
for each task $i \in \{1,\dots,M\}$ **do**
  Init. actor θ_i, buffer $\mathcal{D}_{\text{on}}^i \leftarrow \emptyset$, queue $\mathcal{Q}_i \leftarrow \emptyset$
  // Phase I: Offline Cal-QL Warmup
  Warm up actor θ_i, critic ϕ_i (target ϕ_i') on $\mathcal{D}_{\text{exp}}^i$ via Cal-QL; keep only ϕ_i
  // Phase II: Online Asynchronous Residual Learning
  Launch ACTORPROCESS, LEARNERPROCESS asynchronously
  while not converged **do**
  // Actor process
    if $\mathcal{Q}_i$ is empty **then**
      Generate base-action chunk via $\mu(s_t;\Phi_{\text{base}}^i)$ (eq. (1)); push to $\mathcal{Q}_i$
    end if
    Pop base action $a_{\text{base},t} \leftarrow \mathcal{Q}_i$
    Retrieve active scale $\bar{\Lambda}_{i,t}^{\text{act}}$ and latest synchronized escape signal $s_{\text{escape},u(t)}$
    Compute residual $a_{\text{res},t}$ via eq. (2)
    Form $a_{\text{cand},t}$ via eq. (3)
    Compute $s_{\text{value},t}$ and $\bar{\Lambda}_{i,t+1}^{\text{act}}$ via eqs. (4), (5), (9) and (10)
    if human intervention $a_{\text{human},t}$ detected **then**
      $a_{\text{final},t} \leftarrow a_{\text{human},t}$; $\mathcal{D}_{\text{exp}}^i \leftarrow \mathcal{D}_{\text{exp}}^i \cup \{(s_t, a_{\text{human},t}, a_{\text{base},t}, r_t, s_{t+1})\}$
    else
      $a_{\text{final},t} \leftarrow a_{\text{cand},t}$
    end if
    Store $\mathcal{D}_{\text{on}}^i \leftarrow \mathcal{D}_{\text{on}}^i \cup \{(s_t, a_{\text{final},t}, a_{\text{base},t}, \bar{\Lambda}_{i,t}^{\text{act}}, r_t, s_{t+1})\}$
  // Learner process
    Sample mixed batch $\mathcal{B}_i \sim \text{Mix}(\mathcal{D}_{\text{on}}^i, \mathcal{D}_{\text{exp}}^i)$
    Update critic ϕ_i via twin-Q Bellman objective $\mathcal{L}_Q(\phi_i)$
    Update learner-level $s_{\text{escape},u}$ via eqs. (6) to (8)
    If unlocked, update actor θ_i via eq. (11); lock after K_{lock} consecutive autonomous successes with Q-gain $< \varepsilon_Q$
    Soft-update target ϕ_i'; periodically sync $(\theta_i, \phi_i, s_{\text{escape},u})$ with ACTORPROCESS
  end while
end for
return $\{\theta_i, \phi_i, \mathcal{D}_{\text{on}}^i\}_{i=1}^M$

C. Dynamic Residual Scale

The dynamic residual scale is updated after evaluating the action generated with the currently active scale. At control step t, we denote the composed action in eq. (3) by $a_{\text{cand},t}$ when computing the candidate advantage. Larger scale values strengthen the residual correction, whereas smaller values suppress it.

Term 1: Candidate-Value Modulation (s_{value}). We evaluate whether the currently scaled residual candidate improves the critic estimate relative to the base action:

$$A_i^{\text{cand}}(s_t) = Q_{\phi_i}(s_t, a_{\text{cand},t}) - Q_{\phi_i}(s_t, a_{\text{base},t}). \tag{4}$$

The candidate advantage is mapped through a shifted sigmoid with a non-negative gate:

$$s_{\text{value},t} = 2\max\left(0, \text{sigmoid}\left(\frac{A_i^{\text{cand}}(s_t)}{T}\right) - 0.5\right), \tag{5}$$

where T controls the activation steepness. Thus, the value-driven component is inactive when the currently scaled residual candidate has non-positive estimated gain; the escape component may still activate the overall scale update.

Term 2: Value-Based Stagnation Escape (s_{escape}). Because the learned critic can make the candidate-value gate overly conservative, we additionally monitor learner-level critic progress. We use u to index learner updates, distinguishing them from the control-step index t. Over a lookback window of W learner updates,

$$g_u = \bar{Q}_u - \bar{Q}_{u-W}, \tag{6}$$

where $\bar{Q}_u = \mathbb{E}_{\mathcal{B}_u}[Q_{\phi_i}(s,a)]$ is the mean critic value over the mini-batch used at learner update u. A moving reference is maintained as

$$\hat{g}_u = \eta_g g_u + (1-\eta_g)\hat{g}_{u-1}, \tag{7}$$

where η_g is the smoothing coefficient. The escape signal is

$$s_{\text{escape},u} = \max(0, \max(0, \hat{g}_u) - g_u). \tag{8}$$

The signal becomes positive when recent batch-level critic progress falls below its moving reference, thereby increasing the residual scale through eq. (9) and strengthening corrective intervention when learning progress slows.

Let $u(t)$ denote the latest learner update synchronized to the actor at control step t. The raw scale update is

$$\Lambda^{\text{raw}}_{i,t+1} = \text{clip}\big(\alpha_v s_{\text{value},t} + \alpha_e s_{\text{escape},u(t)}, 0, \lambda_{\max}\big), \tag{9}$$

where α_v and α_e control the contributions of the candidate-value and escape terms. The scale applied at the next control step is obtained through EMA smoothing:

$$\bar{\Lambda}^{\text{act}}_{i,t+1} = \eta_\Lambda \Lambda^{\text{raw}}_{i,t+1} + (1-\eta_\Lambda)\bar{\Lambda}^{\text{act}}_{i,t}, \tag{10}$$

where η_Λ controls the inter-step smoothing. This ordering evaluates the current scaled candidate first and then updates the scale for the subsequent control step, avoiding a circular definition of Λ.

D. Unified IL–RL Optimization

To exploit both the supervised signal from human demonstrations and the self-improvement capability of online RL, we propose a **Unified IL–RL Optimization Framework**. We unify both objectives within $\mathcal{L}_{\text{res}}$ via an adaptive IL mixing coefficient $\rho(t) \in [0,1]$ [16] that smoothly transitions from imitation-dominated to RL-dominated optimization:

$$\mathcal{L}_{\text{res}}(\theta_i) = \rho(t)\mathcal{L}_{\text{IL}} + (1-\rho(t))\mathcal{L}_{\text{SAC}}. \tag{11}$$

Here $\mathcal{L}_{\text{IL}}$ is the imitation-learning loss and $\mathcal{L}_{\text{SAC}}$ is the Soft Actor-Critic (SAC) reinforcement-learning loss, both defined below.

The imitation learning loss drives the residual policy to imitate the expert correction $\delta^* = a_h - \mu(s;\Phi_{\text{base}})$:

$$\mathcal{L}_{\text{IL}} = \mathbb{E}_{(s,\delta^*)\sim\mathcal{D}_i^{\text{exp}}}\big[\|\bar{\Lambda}^{\text{act}}_i(s)\,\pi(s;\theta_i) - \delta^*\|_2^2\big] \tag{12}$$

The expert buffer $\mathcal{D}_i^{\text{exp}}$ contains both offline teleoperation demonstrations and online HIL correction frames. The latter provide supervision at failure-prone states encountered by the current policy and may therefore complement states absent from the initial demonstrations.

The RL objective adopts the SAC framework:

$$\mathcal{L}_{\text{SAC}} = \mathbb{E}_{s\sim\mathcal{B}_i}\left[\alpha_{\text{ent}} \log \pi(\cdot \mid s;\theta_i) - \min_{j\in\{1,2\}} Q_{\phi_i^j}(s, \tilde{a}_i(s))\right]. \tag{13}$$

where α_{ent} is the adaptive entropy temperature and $\tilde{a}_i(s) = a_{\text{base}} + \bar{\Lambda}^{\text{act}}_i(s)\,\tilde{\pi}(s;\theta_i)$ is the composed action, with $a_{\text{base}} = \mu(s;\Phi_{\text{base}})$ the base action and $\tilde{\pi}(s;\theta_i)$ the reparameterized residual sample drawn from the current policy. Following the RLPD paradigm [17], each mini-batch $\mathcal{B}_i$ is composed of 50% online transitions and 50% samples from $\mathcal{D}_i^{\text{exp}}$, enabling Q_{ϕ_i} to learn simultaneously from online exploration and high-quality offline data. We further adopt an update-to-data (UTD) ratio of 4 with DrQ-style image augmentation [18] to stabilize critic learning from visual observations. The active residual scale is treated as a non-differentiable control variable during actor optimization. Equation (11) defines the residual actor objective. The twin critics are optimized separately using the standard SAC Bellman regression objective

$$\mathcal{L}_Q(\phi_i) = \sum_{j\in\{1,2\}} \mathbb{E}_{(s,a,r,s')\sim\mathcal{B}_i}\big[\big(Q_{\phi_i^j}(s,a) - y_t\big)^2\big], \tag{14}$$

where y_t is the shared twin-Q regression target computed from the soft-updated target critics.

The reward signal adopts a sparse success bonus with a step penalty:

$$r_t = \mathbf{1}[\text{success}] - c \tag{15}$$

where $c > 0$ discourages unnecessarily long or stalled trajectories.

We introduce a conditional actor update mechanism: the actor update is *locked* when the policy achieves K_{lock} consecutive autonomous successes *without* human intervention, and the Q-value improvement rate over the last evaluation window falls below threshold ε_Q, jointly indicating that further updates are unlikely to yield behavioral gains.

Optimization comprises two phases.

Phase 1 (Cal-QL Critic Warmup): We warm up the critic on $\mathcal{D}_i^{\text{exp}}$ using Cal-QL [19] and retain the resulting critic weights for online initialization.

Phase 2 (Online Training): actor and critic are updated jointly via eq. (11), where $\rho(t)$ follows a cosine decay from $\rho_{\max} = 1$ to $\rho_{\min}$. The initial pure behavior-cloning (BC) phase ($\rho = 1$) serves as an implicit actor initialization, progressively yielding to $\mathcal{L}_{\text{SAC}}$ as training proceeds:

$$\rho(t) = \rho_{\min} + \frac{\rho_{\max} - \rho_{\min}}{2}\left(1 + \cos\left(\pi \cdot \frac{t - T_{\text{warm}}}{T_{\text{decay}}}\right)\right) \tag{16}$$

for $t \in [T_{\text{warm}}, T_{\text{warm}} + T_{\text{decay}}]$, where T_{warm} and T_{decay} denote the warmup end and decay horizon; $\rho(t)$ is clipped to $\rho_{\max}$ before this interval and $\rho_{\min}$ afterward.

E. Dynamic Feature Modulation and Fusion

To counteract temporal drift and execution inaccuracies in VLA-based action chunking, we introduce an **intra-chunk**

progress-aware orchestration mechanism. This framework employs a residual policy to asynchronously modulate open-loop VLA trajectories, balancing high-level semantic priors with real-time multimodal feedback (RGB-D, force, and intention) for coarse-to-fine control.

To disambiguate execution stages within a chunk, we map the intra-chunk index $n \in \{0,\dots,N-1\}$ to a continuous latent space: $\mathbf{f}_{\text{step}} = \text{MLP}([\text{PE}(n) \parallel n/N])$, where $\text{PE}(\cdot)$ is a sinusoidal positional encoding (PE). For spatial grounding, the geometric intention $\mathbf{f}_{\text{int}}$ encodes the discrepancy between the base-action position p_n^{base} and the current end-effector position p_t^{ee}, i.e., $\Delta p_n = p_n^{\text{base}} - p_t^{\text{ee}}$, by decomposing it into direction and magnitude:

$$\mathbf{f}_{\text{int}} = \text{MLP}([\text{Norm}(\Delta p_n) \parallel \|\Delta p_n\|_2 \parallel \Delta p_n]). \tag{17}$$

For contact-rich interaction, high-frequency force-torque sequences $\mathbf{H}_{f,t}$ are processed via a one-dimensional convolutional neural network (1D-CNN) [20] after mean-centering to remove static bias: $\mathbf{f}_{\text{force}} = \text{1D-CNN}(\mathbf{H}_{f,t} - \bar{\mathbf{H}}_{f,t})$. Concurrently, an RGB-D stream $\mathbf{I}_t$ provides 3D spatial awareness. We employ a ResNet-18 [21] backbone with spatial coordinate convolution (CoordConv) to extract dense visual features $\mathbf{f}_{\text{vis}} = \mathcal{E}_{\text{vis}}(\mathbf{I}_t)$. The depth maps are estimated from RGB using Depth Anything V2 [15].

We utilize FiLM to gate modal features via a temporal anchor $\mathbf{f}_{\text{step}}$. For each modality $m \in \{\text{vis}, \text{int}, \text{force}\}$, the modulated representation is:

$$\text{FiLM}(\mathbf{f}_{\text{step}}, \mathbf{f}_m) = \gamma(\mathbf{f}_{\text{step}}) \odot \mathbf{f}_m + \beta(\mathbf{f}_{\text{step}}). \tag{18}$$

The visual stream $\mathbf{f}'_{\text{vis}} = \text{LayerNorm}(\mathbf{f}_{\text{vis}} + \text{FiLM}(\mathbf{f}_{\text{step}}, \mathbf{f}_{\text{vis}}))$ leverages a residual structure to refine pre-trained spatial representations without overwriting them; the intention and force streams are modulated analogously, $\mathbf{f}'_m = \text{LayerNorm}(\mathbf{f}_m + \text{FiLM}(\mathbf{f}_{\text{step}}, \mathbf{f}_m))$ for $m \in \{\text{int}, \text{force}\}$. The final unified latent $\mathbf{z}_t \in \mathbb{R}^{512}$ is synthesized via late fusion:

$$\mathbf{z}_t = \text{MLP}([\mathbf{f}'_{\text{vis}} \parallel \mathbf{f}_{\text{step}} \parallel \mathbf{f}'_{\text{int}} \parallel \mathbf{f}'_{\text{force}}]). \tag{19}$$

F. Adaptive Residual-Guided Multi-Task VLA Adaptation

Data Preparation and Executed-Scale Normalization. Starting from the shared VLA obtained through mixed-task SFT, we train one task-specific residual policy for each task and collect successful trajectories using the corresponding composed base–residual policy. For the k-th transition of task i, let $\lambda_{i,k}^{\text{exec}}$ denote the active scale actually used to generate its executed composed action. To normalize scale distributions across tasks, we compute the task-wise 20th and 80th percentiles, denoted by $q_{20,i}$ and $q_{80,i}$:

$$\lambda_{i,k}^{\text{norm}} = \text{clip}\left(\frac{\lambda_{i,k}^{\text{exec}} - q_{20,i}}{q_{80,i} - q_{20,i} + \varepsilon}, 0, 1\right). \tag{20}$$

This transformation maps the central range of each task's executed-scale distribution to $[0,1]$ while clipping the lower and upper tails.

Residual-Aware Curriculum Learning. To facilitate efficient policy refinement, we propose a dynamic weighting mechanism that prioritizes samples based on their relative complexity. For the k-th sample in task i, the raw training weight $\omega_{i,k}^{\text{raw}}$ is computed using a Gaussian kernel centered at the curriculum-dependent peak $\Lambda_{\text{peak}}(v)$, where v denotes the weighted-SFT optimization step:

$$\omega_{i,k}^{\text{raw}} = \exp\left(-\frac{\left(\lambda_{i,k}^{\text{norm}} - \Lambda_{\text{peak}}(v)\right)^2}{2\sigma^2}\right). \tag{21}$$

To ensure training stability, weights are upper-bounded by $\omega'_{i,k} = \min(\omega_{i,k}^{\text{raw}}, \omega_{\max})$. Adhering to the curriculum learning paradigm, Λ_{peak} is monotonically increased throughout the training phase. This scheduling enables the agent to initially master fundamental behaviors (low Λ_{peak}) before progressively tackling challenging, high-variance scenarios (high Λ_{peak}).

Balanced Batch Weight Regularization. To maintain task-invariant learning, we construct a balanced batch $\mathcal{B}$ by drawing B/M samples per task. To preserve consistent gradient magnitudes across varying weight distributions, we perform batch-wise normalization for each sample $b \in \mathcal{B}$:

$$\hat{\omega}_b = \frac{\omega'_b}{\frac{1}{B}\sum_{j=1}^{B} \omega'_j + \varepsilon} \tag{22}$$

where ω'_b represents the individual sample weight, B denotes the batch size, and ε is a small constant introduced for numerical stability.

Policy Optimization via Weighted Flow Matching. Based on the normalized weights, the base VLA policy $\mu_{\Phi_{\text{vla}}}$ is fine-tuned using a supervised Flow Matching objective. For each sample $b \in \mathcal{B}$, we designate the successful action a_{final} as the target to construct a conditional probability path. The total loss $\mathcal{L}$ is defined as the weighted empirical expectation of the Flow Matching mean-squared error (MSE) in the velocity space:

$$\mathcal{L} = \mathbb{E}_{b\sim\mathcal{B}}\left[\hat{\omega}_b \left\| v_{\Phi_{\text{vla}}}(x_u, u, c) - v^*(x_u, a_{\text{final}}) \right\|_2^2\right] \tag{23}$$

where $v_{\Phi_{\text{vla}}}$ represents the learnable vector field, x_u is the coupled state at flow-matching time $u \in [0,1]$, and c denotes the native VLA conditioning inputs, including the RGB observation, robot state, and language instruction. The auxiliary depth and force inputs, residual-policy features, and executed residual scale are used only during rollout generation or sample weighting and are not required by the consolidated VLA at inference. The term $\hat{\omega}_b$ denotes the dynamic weight derived from our residual-aware curriculum. By minimizing $\mathcal{L}$, we update the parameters Φ_{vla} to obtain the optimized policy π'_{VLA}.

The complete procedure of section III-F is summarized in algorithm 2.

IV. Experiments

We evaluate RRRR in both real-world and simulation settings. Four Franka manipulation tasks—Weight Insertion, Tissue Extraction, Balance Placement, and Toy Suspension—assess rapid single-task adaptation and controlled multi-task consolidation. A 25-task LIBERO joint-training benchmark

Algorithm 2 Adaptive Residual-Guided Multi-Task VLA Adaptation

Require: Datasets $\{\mathcal{D}_{\text{on}}^i\}_{i=1}^M$, Φ_{vla}, σ, ε, $\omega_{\max}$, $\Lambda_{\text{peak}}(v)$
Ensure: Fine-tuned policy π'_{VLA} with params $\Phi_{\text{vla}}^{final}$
// I. Data Preparation and Residual Analysis
Collect successes: $\mathcal{D}_{\text{succ}}^i \leftarrow \{\zeta \in \mathcal{D}_{\text{on}}^i \mid \text{is_success}(\zeta)\},\ \forall i$
for each task $i \in \{1,\dots,M\}$ **do**
  Compute $q_{20,i}, q_{80,i}$ from $\lambda_{i,k}^{\text{exec}}$ in $\mathcal{D}_{\text{succ}}^i$
  Normalize $\lambda_{i,k}^{\text{norm}}$ via eq. (20)
end for
// II. Weighted Flow Matching Optimization
while not converged **do**
  Update curriculum peak Λ_{peak} by training progress
  for each task $i \in \{1,\dots,M\}$ **do**
    Sample B/M transitions $\{(s, a_{\text{final}}, \lambda_{i,k}^{\text{norm}})\} \sim \mathcal{D}_{\text{succ}}^i$; add to $\mathcal{B}$
  end for
  for each sample $b \in \mathcal{B}$ **do**
    $\omega_b^{\text{raw}} \leftarrow \exp\left(-\frac{(\lambda_b^{\text{norm}} - \Lambda_{\text{peak}}(v))^2}{2\sigma^2}\right)$; $\omega'_b \leftarrow \min(\omega_b^{\text{raw}}, \omega_{\max})$
  end for
  for each sample $b \in \mathcal{B}$ **do**
    $\hat{\omega}_b \leftarrow \frac{\omega'_b}{\frac{1}{B}\sum_{j=1}^{B} \omega'_j + \varepsilon}$
  end for
  $\Phi_{\text{vla}} \leftarrow \Phi_{\text{vla}} - \eta \nabla_{\Phi_{\text{vla}}} \mathcal{L}$ using eq. (23)
end while
return $\Phi_{\text{vla}}^{final} \leftarrow \Phi_{\text{vla}}$

evaluates the complete pipeline at a larger task scale against related RL-based data-generation methods, while an experiment on the ZERITH H1 wheeled bimanual humanoid platform [7] probes cross-embodiment applicability.

Implementation details. All base policies use parameter-efficient low-rank adaptation (LoRA) [22]. Each task-specific backbone in Table I is LoRA-fine-tuned on 20 teleoperated demonstrations, while the shared real-world backbone uses the 80 mixed demonstrations described in section IV-C. A lightweight binary reward model trained on human-labeled rollouts provides both the sparse training reward and automatic success evaluation. Unless otherwise noted, all other settings follow the defaults of the respective base models.

A. Rapid Single-Task Adaptation via Residual Learning

Experimental results show that RRRR enables rapid real-world adaptation of VLA policies. On the OpenVLA-OFT backbone (Table I), the base policy fails on all tasks (0% success), while introducing the residual policy increases the average success rate to 91.25% with minimal interaction. Several tasks require only minutes of training; for example, Weight Insertion and Tissue Extraction achieve 100% success after 3.15 and 4.35 minutes of residual learning, respectively, demonstrating rapid compensation for real-world control errors. As illustrated in fig. 2, the combined VLA+residual policy produces smoother, more goal-directed trajectories, whereas the base VLA exhibits persistent positional drift.

On the $\pi_{0.5}$ backbone, the base policy shows partial capability on some tasks but still benefits substantially from residual correction. For instance, Weight Insertion improves from 5% to 100%, and Toy Suspension from 10% to 70%.

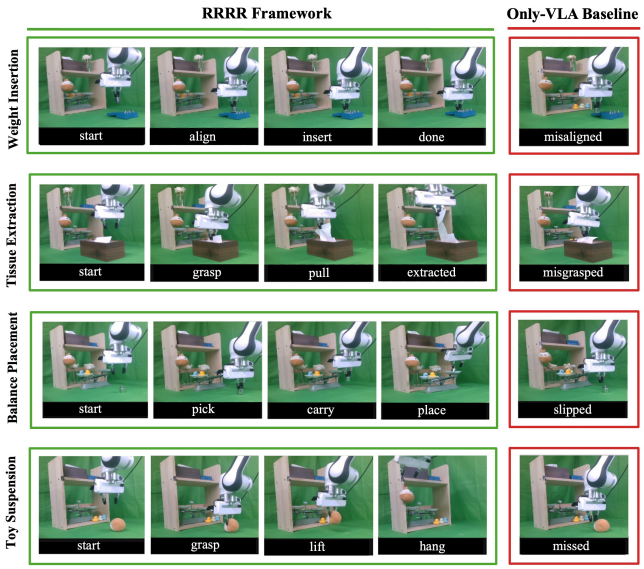


Fig. 2. Qualitative rollout comparisons between independently task-fine-tuned VLAs with and without the proposed residual policy on four real-world tasks.

TABLE I
RAPID RESIDUAL ADAPTATION OF TASK-SPECIFIC LORA-SFT VLA POLICIES. SR DENOTES SUCCESS RATE; TIME DENOTES RESIDUAL-TRAINING WALL-CLOCK TIME TO THE EVALUATED CHECKPOINT.

Task	SR (%)			Time (min)
	Task-Specific LoRA-SFT VLA	+ Residual	Δ	
(a) OpenVLA-OFT [23] as Baseline Model				
Weight Insertion	0 (0/20)	100 (20/20)	+100	3.15
Tissue Extraction	0 (0/20)	100 (20/20)	+100	4.35
Balance Placement	0 (0/20)	95 (19/20)	+95	16.41
Toy Suspension	0 (0/20)	70 (14/20)	+70	27.30
Average	0.00	91.25	+91.25	12.80
(b) $\pi_{0.5}$ [5] as Baseline Model				
Weight Insertion	5 (1/20)	100 (20/20)	+95	10.17
Tissue Extraction	100 (20/20)	100 (20/20)	—	13.14
Balance Placement	65 (13/20)	95 (19/20)	+30	11.45
Toy Suspension	10 (2/20)	70 (14/20)	+60	6.51
Average	45.00	91.25	+46.25	10.32

B. Ablation Study on Residual Design

We ablate the residual design with $\pi_{0.5}$ on two real-world tasks: Pick-and-Place, in which the robot places a plush toy into a plate, and Weight Insertion. Fixed Λ replaces the dynamic scale with a constant; w/o s_{escape} removes stagnation recovery; RL only sets $\rho(t) \equiv 0$; and w/o force and w/o depth remove the force and depth channels, respectively. Within each task, all variants use identical demonstrations, expert buffers, optimization schedules, and online training duration.

Table II separates the components that determine task feasibility from those that primarily affect execution efficiency. Removing imitation guidance produces the largest success

TABLE II

RESIDUAL-DESIGN ABLATION ON TWO REAL-WORLD TASKS ($\pi_{0.5}$ BACKBONE; 30 TRIALS PER CONDITION). EXECUTION TIME IS REPORTED IN SECONDS AS MEAN±STANDARD DEVIATION OVER SUCCESSFUL TRIALS.

Variant	Pick-and-Place		Weight Insertion	
	SR (%)	Exec (s)	SR (%)	Exec (s)
RRRR (full)	**100.0**	$\mathbf{11.80 \pm 0.83}$	**100.0**	$\mathbf{4.78 \pm 1.00}$
Fixed Λ	73.3	23.36 ± 19.36	96.7	6.01 ± 1.45
RL only ($\rho \equiv 0$)	66.7	17.86 ± 3.51	63.3	10.39 ± 2.79
w/o s_{escape}	80.0	15.88 ± 4.61	83.3	10.63 ± 1.68
w/o force	96.7	13.45 ± 1.99	96.7	5.59 ± 1.75
w/o depth	100.0	12.92 ± 0.69	93.3	7.03 ± 2.66

degradation, reducing success by 33.3 and 36.7 points on Pick-and-Place and Weight Insertion, respectively. Eliminating s_{escape} also reduces success on both tasks. A fixed residual scale exhibits marked task dependence: it preserves near-saturated success on Weight Insertion but reduces Pick-and-Place success from 100.0% to 73.3%, indicating that state-independent correction can over-perturb the base policy.

Execution time provides additional discrimination once success approaches saturation. Every ablation increases the mean completion time. Removing force or depth largely preserves success (93.3–100.0%) but increases execution time by 9–47%, suggesting that these modalities primarily improve execution efficiency. Fixed Λ also yields substantially greater execution-time variability on Pick-and-Place (23.36 ± 19.36 s versus 11.80 ± 0.83 s), highlighting the role of state-dependent scaling in maintaining consistent execution.

C. Residual-Guided Multi-Task VLA Adaptation

1) Real-World Evaluation on Franka: We further evaluate the proposed residual-guided multi-task update mechanism on a Franka robot across four manipulation tasks, reporting success rates over 20 trials per task. Using $\pi_{0.5}$ as the backbone, we first train a shared VLA on 80 mixed demonstrations (20 per task), denoted as *Base (Mixed SFT)*. We then compare this common base model with three adaptation variants.

Base + Residual$_i$ freezes the shared base VLA and trains one RL residual policy for each task T_i. Its reported success rate is measured under composed base–residual execution, with the corresponding task-specific residual retained at inference. Weighted SFT fine-tunes the shared base VLA on 80 successful composed-policy rollouts (20 per task) using the proposed residual-scale-derived sample weights. The *w/o Weighting* ablation uses exactly the same rollouts and optimization settings as Weighted SFT but replaces the residual-scale-derived weights with uniform weights.

As shown in Table III, Base (Mixed SFT) achieves 11.25% average success. Fine-tuning the residual-generated rollouts with uniform weights (*w/o Weighting*) raises average success to 23.75%, whereas applying residual-scale weights to the same rollouts raises it to 85.00%, directly isolating the benefit of the proposed weighting. *Base + Residual$_i$* achieves 90.00% through task-specific residual correction, but retains a separate residual policy at inference; Weighted SFT instead consolidates the interaction experience into one standalone shared VLA.

TABLE III

MULTI-TASK SUCCESS RATES (%) ON (A) FRANKA AND (B) LIBERO-25. BOLD: BEST STANDALONE MODEL IN (A) AND BEST OVERALL METHOD IN (B).

(a) Real-world Franka evaluation (20 trials per task)

Task	Base (Mixed SFT)	Base + Residual$_i$	Weighted SFT	w/o Weighting
Weight Insertion	0	100	**100**	55
Tissue Extraction	45	100	**90**	0
Balance Placement	0	85	**70**	30
Toy Suspension	0	75	**80**	10
Average	11.25	90.00	**85.00**	23.75

(b) Custom LIBERO-25 joint-training evaluation (200 episodes per task)

Method	LIBERO-10 10 tasks	LIBERO-90 15 tasks	Overall
DPRL [13]	84.40	85.67	85.16
PLD [14]	83.30	90.33	87.52
RRRR (Weighted SFT)	**87.15**	**94.90**	**91.80**

2) Large-Scale Evaluation on LIBERO: To evaluate the scalability of the multi-task component, we construct a custom 25-task joint-training setting from LIBERO [24]. We first obtain a shared $\pi_{0.5}$ VLA through mixed-task SFT, directly using the demonstrations provided by LIBERO for the selected tasks. Task-specific residual policies are then trained on top of this frozen shared VLA and used to collect 50 successful composed-policy rollouts per task, which are finally consolidated through residual-guided weighted SFT. The benchmark comprises all 10 long-horizon tasks from LIBERO-10 and a fixed subset of 15 tasks from LIBERO-90. The LIBERO-90 subset spans three scenes—Kitchen (6), Living Room (5), and Study (4 tasks)—selected for multi-step behaviors, spatially precise manipulation, and object diversity without exact duplicates. All 25 tasks are jointly used for multi-task post-training. For a controlled comparison, all methods share the same backbone, task split, optimization schedule, and evaluation protocol, and all data-generation methods use the same budget of 50 generated trajectories per task.

We compare RRRR with two RL-based data-generation approaches. DPRL [13] uses diffusion-based RL to generate trajectories for subsequent VLA training. PLD [14] uses a state-independent residual bound and standard unweighted SFT, providing a residual-RL-to-SFT baseline without dynamic scaling or scale-guided weighting. We do not evaluate RLDG [12] because its specialist stage requires real-robot human-in-the-loop corrections outside the LIBERO protocol.

RRRR achieves 87.15% on LIBERO-10 and 94.90% on the selected LIBERO-90 tasks, yielding 91.80% overall and



Fig. 3. Snack-Retrieval sequence on ZERITH H1: start, grasp, handover, and completion.

outperforming DPRL and PLD by 6.64 and 4.28 percentage points, respectively.

Cross-embodiment applicability. We further deploy RRRR on ZERITH H1 [7], a wheeled bimanual humanoid platform. The snack-retrieval task spans grasping, inter-arm handover, and placement: the right arm retrieves a snack from the shelf, passes it to the left arm, and the left arm places it in a basket (fig. 3). We retain the frozen base VLA and train only the task-specific residual policy under the same action-composition framework. Across 20 trials, success rises from 0% for the base policy to 75% (15/20) with residual adaptation, showing applicability beyond the single-arm Franka setup without updating the base VLA.

V. Conclusion

We presented RRRR for two complementary VLA post-training settings. First, a high-frequency residual policy adapts an independently task-tuned VLA while keeping the backbone frozen. Second, task-specific residual policies collect successful interaction trajectories on a shared mixed-SFT backbone, and the recorded executed scales guide weighted consolidation into one standalone shared VLA.

On four real-world tasks, residual adaptation raises the average success rates of task-specific OpenVLA-OFT and $\pi_{0.5}$ backbones from 0.00% and 45.00% to 91.25%, with the evaluated checkpoints reached after 3.15–27.30 min of residual training in our setup. For shared multi-task adaptation, Weighted SFT achieves 85.00% versus 23.75% for *w/o Weighting* on identical rollouts, while RRRR reaches 91.80% in the custom LIBERO-25 setting. These results support short-wall-clock residual adaptation and shared-model consolidation at a larger task scale.

The real-world study remains limited to four primary Franka tasks, with the single-task H1 experiment providing only preliminary cross-embodiment evidence. The reported training times measure wall-clock adaptation speed rather than environment-step sample efficiency, and Table I does not compare against direct online optimization of VLA parameters. Future work will investigate shared residual representations, broader embodiments, and adaptation with less task-specific interaction.

References

[1] A. Brohan, N. Brown, J. Carbajal, Y. Chebotar, J. Dabis, C. Finn, K. Gopalakrishnan, K. Hausman, A. Herzog, J. Hsu, *et al.*, "Rt-1: Robotics transformer for real-world control at scale," *arXiv preprint arXiv:2212.06817*, 2022.

[2] B. Zitkovich, T. Yu, S. Xu, P. Xu, T. Xiao, F. Xia, J. Wu, P. Wohlhart, S. Welker, A. Wahid, *et al.*, "Rt-2: Vision-language-action models transfer web knowledge to robotic control," in *Conference on Robot Learning*. PMLR, 2023, pp. 2165–2183.

[3] M. J. Kim, K. Pertsch, S. Karamcheti, T. Xiao, A. Balakrishna, S. Nair, R. Rafailov, E. Foster, G. Lam, P. Sanketi, *et al.*, "Openvla: An open-source vision-language-action model," *arXiv preprint arXiv:2406.09246*, 2024.

[4] K. Black, N. Brown, D. Driess, A. Esmail, M. Equi, C. Finn, N. Fusai, L. Groom, K. Hausman, B. Ichter, *et al.*, "π0: A vision-language-action flow model for general robot control," *arXiv preprint arXiv:2410.24164*, 2024.

[5] P. Intelligence, K. Black, N. Brown, J. Darpinian, K. Dhabalia, D. Driess, A. Esmail, M. Equi, C. Finn, N. Fusai, *et al.*, "π0.5: A vision-language-action model with open-world generalization," *arXiv preprint arXiv:2504.16054*, 2025.

[6] O. M. Team, D. Ghosh, H. Walke, K. Pertsch, K. Black, O. Mees, S. Dasari, J. Hejna, T. Kreiman, C. Xu, *et al.*, "Octo: An open-source generalist robot policy," *arXiv preprint arXiv:2405.12213*, 2024.

[7] Zerith Robotics, "ZERITH H1: Wheeled-arm robot," 2025. [Online]. Available: https://www.zerith.ai/arm

[8] L. Ankile, A. Simeonov, I. Shenfeld, M. Torne, and P. Agrawal, "From imitation to refinement-residual rl for precise assembly," in *2025 IEEE International Conference on Robotics and Automation (ICRA)*. IEEE, 2025, pp. 01–08.

[9] L. Ankile, Z. Jiang, R. Duan, G. Shi, P. Abbeel, and A. Nagabandi, "Residual off-policy rl for finetuning behavior cloning policies," *arXiv preprint arXiv:2509.19301*, 2025.

[10] X. Xu, Y. Hou, C. Xin, Z. Liu, and S. Song, "Compliant residual dagger: Improving real-world contact-rich manipulation with human corrections," *arXiv preprint arXiv:2506.16685*, 2025.

[11] J. Luo, C. Xu, J. Wu, and S. Levine, "Precise and dexterous robotic manipulation via human-in-the-loop reinforcement learning," *Science Robotics*, vol. 10, no. 105, p. eads5033, 2025.

[12] C. Xu, Q. Li, J. Luo, and S. Levine, "Rldg: Robotic generalist policy distillation via reinforcement learning," *arXiv preprint arXiv:2412.09858*, 2024.

[13] R. Yang, H. Wei, R. Zhang, Z. Feng, X. Chen, T. Li, C. Zhang, L. Zhao, J. Bian, X. Su, and Y. Chen, "Beyond human demonstrations: Diffusion-based reinforcement learning to generate data for vla training," 2025. [Online]. Available: https://arxiv.org/abs/2509.19752

[14] W. Xiao, H. Lin, A. Peng, H. Xue, T. He, Y. Xie, F. Hu, J. Wu, Z. Luo, L. Fan, *et al.*, "Self-improving vision-language-action models with data generation via residual rl," in *International Conference on Learning Representations*, 2026.

[15] L. Yang, B. Kang, Z. Huang, Z. Zhao, X. Xu, J. Feng, and H. Zhao, "Depth anything v2," *ArXiv*, vol. abs/2406.09414, 2024.

[16] W. Zhang, Y. Xie, Y. Sun, Y. Chen, G. Wang, Y. Li, B. Ding, and J. Zhou, "On-policy rl meets off-policy experts: Harmonizing supervised fine-tuning and reinforcement learning via dynamic weighting," in *International Conference on Learning Representations*, 2026.

[17] P. J. Ball, L. Smith, I. Kostrikov, and S. Levine, "Efficient online reinforcement learning with offline data," in *International Conference on Machine Learning*, 2023, pp. 1577–1594.

[18] D. Yarats, I. Kostrikov, and R. Fergus, "Image augmentation is all you need: Regularizing deep reinforcement learning from pixels," in *International Conference on Learning Representations*, 2021.

[19] M. Nakamoto, Y. Zhai, A. Singh, M. S. Mark, Y. Ma, C. Finn, A. Kumar, and S. Levine, "Cal-ql: Calibrated offline rl pre-training for efficient online fine-tuning," in *Advances in Neural Information Processing Systems*, 2023.

[20] S. Kiranyaz, O. Avcı, O. Abdeljaber, T. Ince, M. Gabbouj, and D. J. Inman, "1d convolutional neural networks and applications: A survey," *ArXiv*, vol. abs/1905.03554, 2019.

[21] K. He, X. Zhang, S. Ren, and J. Sun, "Deep residual learning for image recognition," *2016 IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pp. 770–778, 2015.

[22] E. J. Hu, Y. Shen, P. Wallis, Z. Allen-Zhu, Y. Li, S. Wang, L. Wang, and W. Chen, "Lora: Low-rank adaptation of large language models," in *International Conference on Learning Representations*, 2022.

[23] M. J. Kim, C. Finn, and P. Liang, "Fine-tuning vision-language-action models: Optimizing speed and success," *arXiv preprint arXiv:2502.19645*, 2025.

[24] B. Liu, Y. Zhu, C. Gao, Y. Feng, Q. Liu, Y. Zhu, and P. Stone, "Libero: Benchmarking knowledge transfer for lifelong robot learning," *arXiv preprint arXiv:2306.03310*, 2023.